جـامعة فلســــــطـين التقنيـــة - خضـــــــــوري

Palestine Technical University – Khadoorie

Information Technology and Artificial Intelligence

Computer Science

Graduation Project

Interactive Food Ordering System

By:

Laila Salman

Tasbeeh Yadak

Hana Yasin

Sadeel Saleh

Supervisor:

Dr. Hasan Zakaria Alhajhamad

2025-2026

I dedicate this humble work to my beloved parents.
To my father, who has always been my source of strength and support, and who gave me everything without hesitation.
To my precious mother, whose compassion, kindness, and endless encouragement taught me how to stand firm and face every challenge in life.

I also dedicate this work to my brothers, sisters, and friends, whose support and motivation have always been a source of inspiration.
To my family and to everyone who contributed to shaping my character and lighting the path before me, this work is sincerely dedicated.

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude and appreciation to my supervisor, Dr. Hasan Z. H. Alhajhamad, for his continuous support, guidance, and valuable feedback throughout the development of this graduation project.

His expertise, encouragement, and constructive advice played a significant role in shaping this work from its early stages until completion.

I would also like to extend my heartfelt thanks to all the doctors and instructors in the Applied computing department for their knowledge, support, and dedication during my academic journey.

ABSTRACT

This project presents a smart food ordering and delivery system that aims to improve restaurant operations and enhance customer experience through a unified digital platform. The system connects customers, restaurants, and delivery drivers, allowing users to place orders, track deliveries in real time, and receive personalized food recommendations. Traditional food delivery methods often suffer from delays, inaccurate delivery time estimation, poor coordination between restaurants and drivers, and limited order tracking. To address these issues, the proposed system integrates modern web technologies and artificial intelligence techniques to provide accurate delivery time prediction, intelligent driver routing, and AI-based restaurant recommendations. The main objective of this project is to develop an efficient, user-friendly, and scalable food delivery application that reduces operational challenges, improves service quality, and supports local restaurants in adapting to digital transformation.

TABLE OF CONTENTS

Subject	P.No
Dedication	i
Acknowledgements	ii
Abstract	iii
Table of Contents	iv
List of Figures	v
List of Tables	vi
Chapter One: Introduction	
1. Overview	1
2. Background (Related Work)	2
3. Problem Statement	3
4. Project Objectives	3
5. Project Aim	4
6. Problems for a Smart Food Application	4,5
7. Project Solutions	5,6
8. Project Benefits	7
9. Top Competitors	8-11
10. Project Contribution	12
11. Project Limitations	12-13
12. Project Organization	13
Chapter Two: Software Development Model	
1. Selected Software Development Model	14
2. Justification of the Chosen Model	14-15
3. Current System vs. Proposed System	15-16
4.1 Functional Requirements	16
4.2 Non-Functional Requirements	16
5. Use Case Diagram	17
6. Use Case Scenarios	18
Chapter Three: System Design	
1. UML Diagrams	19
1.1 Class Diagram	19-21

Subject	P.No
1.2 Sequence Diagram	22-23
1.3 Activity Diagram	23-24
1.4. Entity Relationship Diagram (ERD)	25-26
2.System Architecture	27-29
3. Database Design	29-30
4. User Interface Design	31-35
5. Development Environment	36-39
6. Tools, Languages, and Frameworks	40-41
Chapter Four: Conclusion	
1. Conclusion	42
References	43
Appendices	
Appendix A: UML Diagrams	43
Appendix B: User Interface Snapshots	43
Appendix C: Database Design	43
Appendix D: Code Samples	44-48

LIST OF FIGURES

Figure 1.1 Logo of Yummy Palestine Food Delivery Application 8

Figure 1.2 Logo of Tawasi Online Food Delivery Platform 9

Figure 1.3 Wolt Food Delivery Service and Delivery Rider 10

Figure 1.4 Logo of Uber Eats Food Delivery Platform 11

Figure 3.1 Class Diagram of the System ..... 19 - 20

Figure 3.2 Sequence Diagram for Food Ordering Process 22

Figure 3.3 Activity Diagram 23

Figure 3.4 Entity Relationship Diagram (ERD) 25

Figure 3.5 System Architecture of the Food Ordering and Delivery System 29

Figure 3.6 Initial User Interface Wireframes for the Customer Application 32

Figure 3.7 Initial Wireframes for Admin and Super Admin Interfaces 33

Figure 3.8 User Interface Wireframes for Restaurant and Kitchen Staff Operations ..35

Figure 3.9 Development Environment Tools and Technologies 39

Figure D.1 SQL Database Creation and Execution Log 44

CHAPTER ONE

INTRODUCTION

1. Overview

With the rapid growth of technology and the widespread use of smartphones, online food ordering and delivery services have become an essential part of modern daily life. Restaurants increasingly rely on digital platforms to manage orders, reach customers, and improve service efficiency. Food delivery applications allow users to browse menus, place orders, make payments, and track deliveries with ease, providing convenience for both customers and restaurant owners.

Despite the availability of several food delivery platforms, many local restaurants still face operational challenges such as inaccurate delivery time estimation, inefficient coordination between restaurants and delivery drivers, limited order tracking, and lack of personalized recommendations. These issues often lead to delays, customer dissatisfaction, and increased operational costs.

The integration of artificial intelligence and modern software technologies offers effective solutions to these challenges. AI techniques can be used to predict delivery time, optimize driver routing, analyze customer behavior, and provide personalized food and restaurant recommendations. Such intelligent systems enhance user experience, improve operational efficiency, and support data-driven decision-making for restaurant management.

This project focuses on designing and developing an interactive and intelligent food ordering and delivery system that connects customers, restaurants, and delivery drivers through a unified platform. The proposed system aims to improve service quality, reduce delivery time, enhance communication between system components, and support local restaurants in adapting to the digital transformation of the food delivery industry.

2. Background (Related Work)

The rapid advancement of information technology and mobile applications has significantly transformed the food service industry. Online food ordering and delivery platforms have become an essential part of modern life, enabling customers to order meals conveniently while allowing restaurants to expand their reach and improve operational efficiency. As a result, food delivery systems have become a competitive and rapidly evolving domain.

In recent years, the growth of food delivery services has been driven by increased smartphone usage, busy lifestyles, and the demand for convenience. To manage this growth, modern platforms rely on digital systems to handle orders, payments, and coordination with delivery drivers. Artificial intelligence has become a key component in these systems, where it is used to predict delivery time, optimize driver routing, analyze customer behavior, and provide personalized food and restaurant recommendations, leading to improved service quality and customer satisfaction.

Several existing food delivery applications such as Talabat, Uber Eats, Careem NOW, offer core functionalities including restaurant browsing, order placement, payment processing, and order tracking. Despite their popularity, many of these platforms still face limitations such as high service fees, limited personalization, and inaccurate delivery time estimation. These limitations highlight the need for a smarter and more efficient food delivery system, which motivates the development of the proposed project.

3. Problem Statement

Many existing food delivery applications suffer from operational and user experience issues such as inaccurate delivery time estimation, limited real-time order tracking, and weak communication between customers, restaurants, and delivery drivers. These problems often result in delays, customer dissatisfaction, and inefficient service management.

Restaurants also face challenges in coordinating food preparation with driver arrival times and managing orders effectively, which increases operational costs. In addition, most current systems lack intelligent recommendation features that personalize food and restaurant suggestions based on user preferences.

These limitations highlight the need for a smart and integrated food delivery system that improves coordination, delivery accuracy, and overall user experience through modern technologies and artificial intelligence.

4. Project Objectives

1. To design and develop a smart food ordering and delivery system that connects customers, restaurants, and delivery drivers through a unified digital platform.
2. To improve delivery efficiency by providing accurate delivery time estimation and real-time order tracking.
3. To enhance user experience through a simple, user-friendly, and interactive interface for customers and administrators.
4. To utilize artificial intelligence techniques to provide personalized food and restaurant recommendations based on user preferences and order history.
5. To support restaurant management by offering administrative dashboards for monitoring orders, customers, and sales performance.

5. Project Aim

The main aim of this project is to develop an interactive and intelligent food ordering and delivery system that improves restaurant operations, enhances delivery efficiency, and provides customers with a convenient and personalized ordering experience.

6. Problems For A Smart Food Application

1) Inaccurate Delivery Time Estimation

Most platforms rely on static average estimates instead of real-time data. They often ignore important factors such as traffic congestion, weather conditions, restaurant preparation delays, and driver speed.

Result: This leads to customer frustration and an increased number of complaints.

2) Lack of Precise Real-Time Tracking

Users cannot clearly determine whether their food is being prepared, when the driver will arrive at the restaurant, or the exact driver location.

Result: This causes uncertainty and negatively affects the overall user experience.

3) Inefficient Driver Routing

Drivers may select longer or slower routes. Many systems do not dynamically update routing based on live traffic data, road closures, or accidents.

Result: Delayed deliveries and higher operational costs.

4) Restaurant Preparation Delays

Restaurants may prepare food without proper coordination with driver arrival time.

Result: Wasted driver time, cold food, and longer delivery times.

5) Poor Order Stacking (Batching Issues)

Drivers may receive multiple far-apart orders at the same time. Inefficient batching leads to late deliveries, higher fuel consumption, and lower driver satisfaction.

6) Weak Communication with Customers

Customers often ask questions such as:

“Has the driver arrived?” or “Is my order being prepared?”

Result: A lack of automatic updates reduces transparency and customer trust.

7) No Smart Restaurant Recommendations

Users spend excessive time searching for food because the system does not consider user preferences, previous orders, cuisine interests, or delivery speed.

Result: Lower user satisfaction and engagement.

7. Project Solutions

1) AI-Powered ETA (Estimated Time of Arrival) Prediction

The system provides accurate delivery time predictions using real-time traffic analysis, restaurant preparation time, weather conditions, driver historical behavior, and live demand data.

Result: More accurate delivery estimates and fewer customer complaints.

2) Real-Time GPS Tracking

A live tracking interface displays order preparation status, driver route, exact location, remaining delivery time, and surrounding traffic conditions.

Result: Full transparency and increased user trust.

3) Smart Driver Routing

The system automatically selects the fastest route with the least traffic, shortest distance, and lowest fuel consumption.

Result: Faster deliveries and reduced operational costs.

4) Synchronized Restaurant–Driver Coordination

AI predicts when the restaurant should start cooking based on the driver's estimated arrival time.

Result: Zero waiting time, hotter food, and faster delivery.

5) Intelligent Order Stacking (Batching)

AI checks:

- which orders can be delivered together
- whether locations align
- whether batching affects ETA

Result: Higher efficiency, higher driver income, shorter delivery times.

The system sends automatic updates:

- "Your food is being prepared."
- "Your driver is on the way."
- "Driver has arrived at the restaurant."
- "Order is almost at your location."

Result: no need for customer–driver calls.

6) AI-Based Restaurant Recommendation System

Uses machine learning to suggest:

- restaurants that match user taste
- meals similar to past preferences
- places with the fastest delivery
- trending meals in the area
- personalized promotions

Result: Better user experience + higher sales for restaurants.

8. Project Benefits

For Customers:

1. Easy and convenient food ordering through a user-friendly application.
2. Real-time order tracking and accurate delivery time estimation.
3. Personalized food and restaurant recommendations based on user preferences.

For Restaurants:

1. Efficient management of incoming orders and food preparation processes.
2. Improved coordination with delivery drivers.
3. Access to analytical dashboards for monitoring sales and performance.

For Delivery Drivers:

1. Optimized route suggestions to reduce delivery time.
2. Clear order details and delivery instructions.

For System Administrators:

1. Centralized management of users, restaurants, and orders.
2. Improved system monitoring and decision-making support.

9. Top Competitors

- **Yummy Palestine:**

Figure 1.1: Logo of Yummy Palestine Food Delivery Application

PROS:

- Food & grocery delivery available.

- user-friendly app,scheduling orders possible.

- good user ratings (~4.3/5).

CONS:

- Some users report service quality issues.

- coverage limited to specific cities.

- smaller infrastructure may cause delays.

- **Tawasi Online:**

Figure 1.2: Logo of Tawasi Online Food Delivery Platform

PROS:

- Connects restaurants, stores, local drivers.

- supports cash & card Payments.

- easy registration for small Businesses.

CONS:

- service quality may vary. - coverage limited to some Areas.

- smaller platform with Less stability.

- **Wolt:**

Figure 1.3: Wolt Food Delivery Service and Delivery Rider

PROS:

- Large restaurant variety & user base.

- real-time order tracking.

- fast delivery in urban areas.

CONS:

- High service fees/commissions.

- quality depends on partners/suppliers.

- coverage not universal.

- **Uber Eats:**

Figure 1.4: Logo of Uber Eats Food Delivery Platform

PROS:

- Wide global network, ideal for expansion.

- integrated order management System.

- large fleet ensures fast Delivery.

CONS:

- High commissions (15–30%) reduce profitability.

- restaurants lose direct customer relationship.

- higher cost for consumers Compared to direct orders.

10. Project Contribution

The proposed smart food delivery system contributes to improving restaurant and delivery operations by providing an integrated and intelligent digital platform. The system combines real-time order tracking, accurate delivery time estimation, and intelligent driver coordination to enhance service efficiency and reduce operational delays.

In addition, the project introduces AI-based recommendation mechanisms that personalize food and restaurant suggestions based on user preferences and order history. This contribution improves user experience, reduces search time, and increases customer satisfaction.

The system also provides administrative dashboards that support data-driven decision-making by offering clear insights into orders, customers, and sales performance. Moreover, the application is designed with a user-friendly interface and scalable architecture, making it suitable for future expansion and adaptation to different restaurant environments.

11. Project Limitations

The development of the smart food delivery system faced several limitations that affected the scope and implementation of the project. One of the main limitations was the limited project development time, which restricted the ability to implement and test all planned features in depth.

Another limitation was the dependency on simulated or limited real-world data, especially for testing artificial intelligence components such as recommendation systems and delivery time prediction. This limitation may affect the accuracy of results compared to large-scale commercial systems.

Additionally, the project focuses on core food ordering and delivery functionalities and does not fully cover advanced features such as real-time traffic integration, multi-payment gateway support, and large-scale scalability. Furthermore, system performance may vary depending on server resources and network availability.

Despite these limitations, the proposed system successfully demonstrates the core concepts of a smart and intelligent food delivery platform and provides a strong foundation for future improvements and extensions.

12. Project Organization

This graduation project is organized into five main chapters, each focusing on a specific aspect of the proposed system.

Chapter One:

introduces the project and provides an overview of the smart food delivery system. It also presents the background and related work, problem statement, project objectives and aim, project benefits, contribution, limitations, and the overall organization of the report.

Chapter Two:

describes the software development model adopted in this project. It explains the selected development model, justifies the choice, compares the current system with the proposed system, and presents the system requirements, including functional and non-functional requirements, as well as use case diagrams and scenarios.

Chapter Three:

focuses on the system design. It includes the system architecture, UML diagrams, entity relationship diagram (ERD), database design, user interface design, development environment, and the tools, languages, and frameworks used in the project.

Chapter Four:

concludes the project by summarizing the main outcomes and discussing the overall achievements of the proposed system.

CHAPTER TWO

SOFTWARE DEVELOPMENT MODEL

1. Selected Software Development Model

The Agile model was used in this project to improve flexibility and teamwork throughout the development process. The project was divided into small iterations, which allowed the team to plan, develop, and review features step by step. Regular meetings helped the team communicate effectively and quickly respond to changes or new requirements. By using Agile, the team was able to deliver high-quality results on time while continuously improving the project based on feedback.

2. Justification of the Chosen Model

1. Flexibility in Development Agile allows the team to adapt quickly to changes in requirements, design, or features. Since food-delivery systems often evolve during development, this flexibility is essential.

2. Continuous Improvement The iterative nature of Agile helps the team improve the application step by step. After every sprint, the team evaluates what worked well and what needs to be improved.

3. Better Team Collaboration Agile encourages frequent communication among team members through daily or weekly meetings. This helps ensure that everyone is aligned, tasks are clear, and problems are solved early.

4. Faster Delivery of Working Features Instead of waiting until the end of the semester to have a full system, Agile allows the team to deliver a working version of the app at the end of each sprint. This makes testing and refinement easier.

5. Early Detection of Issues Frequent reviews and testing sessions help the team discover bugs or design problems early, reducing the risk of major issues near the project deadline.

6. Suitable for Limited Time Projects Since your project has time constraints, Agile provides a structured way to break the work into small, manageable tasks. This makes it easier to stay on schedule.

7. Allows Incremental Addition of Features The team can start with core features (ordering, tracking, restaurant list) and gradually add advanced features (ETA prediction, map integration) based on time and priority.

8. Improves Quality Through Continuous Testing Each sprint ends with reviewing and testing the work done. This ensures the system quality improves continuously throughout the project lifecycle.

3. Current System vs Proposed System

3.1 Current System

Most restaurants currently rely on traditional systems, which include:

- Manual ordering through waiters.
- Paper-based menus.
- No personalization based on customer preferences.
- Limited or no use of customer data analysis.
- Longer waiting times and higher probability of human errors.

3.2 Proposed System

The proposed system is a **smart restaurant mobile application** enhanced with Artificial Intelligence, offering:

- Digital and interactive food menu.
- AI-based personalized food recommendations.

- Online ordering and electronic payment.
- Real-time order tracking.
- Customer data collection and analysis for service improvement.
- Faster service with reduced ordering errors.

4. Functional and Non-Functional Requirements

4.1 Functional Requirements

- User registration and login.
- Display food menu with images, descriptions, and prices.
- Search and filter menu items.
- AI-based food recommendations based on user preferences and order history.
- Add items to cart.
- Place orders and perform online payment.
- Track order status in real time.
- Rate and review meals and services.
- Admin dashboard for restaurant management.

4.2 Non-Functional Requirements

- User-friendly and intuitive interface.
- High system performance and fast response time.
- Secure handling of user data and payment information.
- Compatibility with Android and iOS platforms.
- Scalability to support a large number of users.
- High availability and reliability.

5. Use-Case Diagram

5.1 Main Actors

- Customer
- Restaurant Administrator
- Payment System
- AI Recommendation System

5.2 Use Cases

- Register / Login
- View Menu
- Place Order
- Receive AI Recommendations
- Make Payment
- Track Order
- Manage Menu (Admin)
- View Reports (Admin)

6. Use-Case Scenarios

Scenario 1: Place Order

1. The customer logs into the application.
2. The system displays the food menu.
3. The AI system suggests meals based on customer preferences.
4. The customer selects items and adds them to the cart.
5. The customer confirms the order and completes payment.
6. The system displays the order status for tracking.

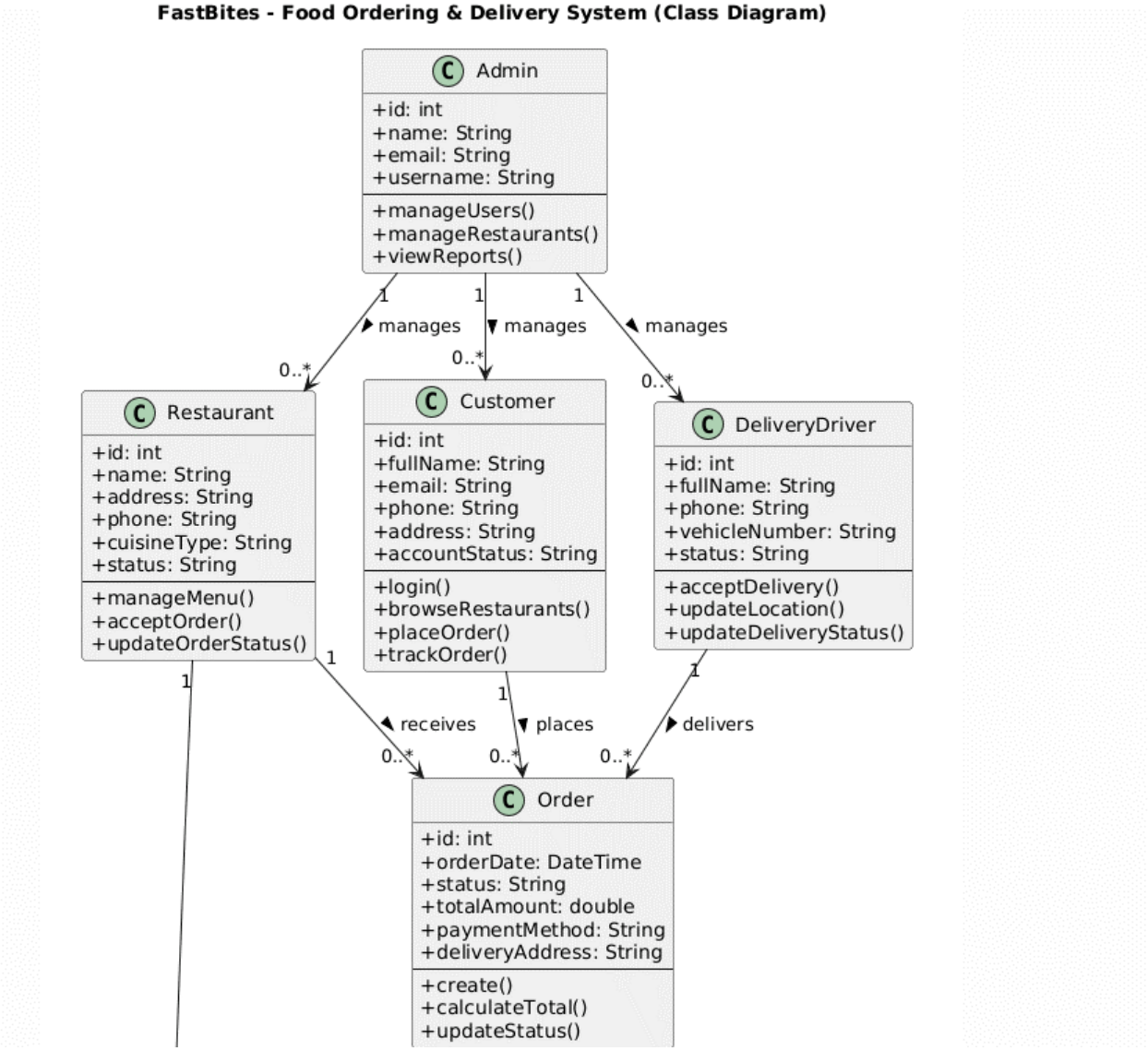
Scenario 2: AI-Based Recommendation

1. The customer opens the application.
2. The system analyzes previous orders and preferences.
3. Personalized meal recommendations are generated.
4. The customer may accept or ignore the recommendations.

CHAPTER THREE

SYSTEM DESIGN

1.1 Class Diagram



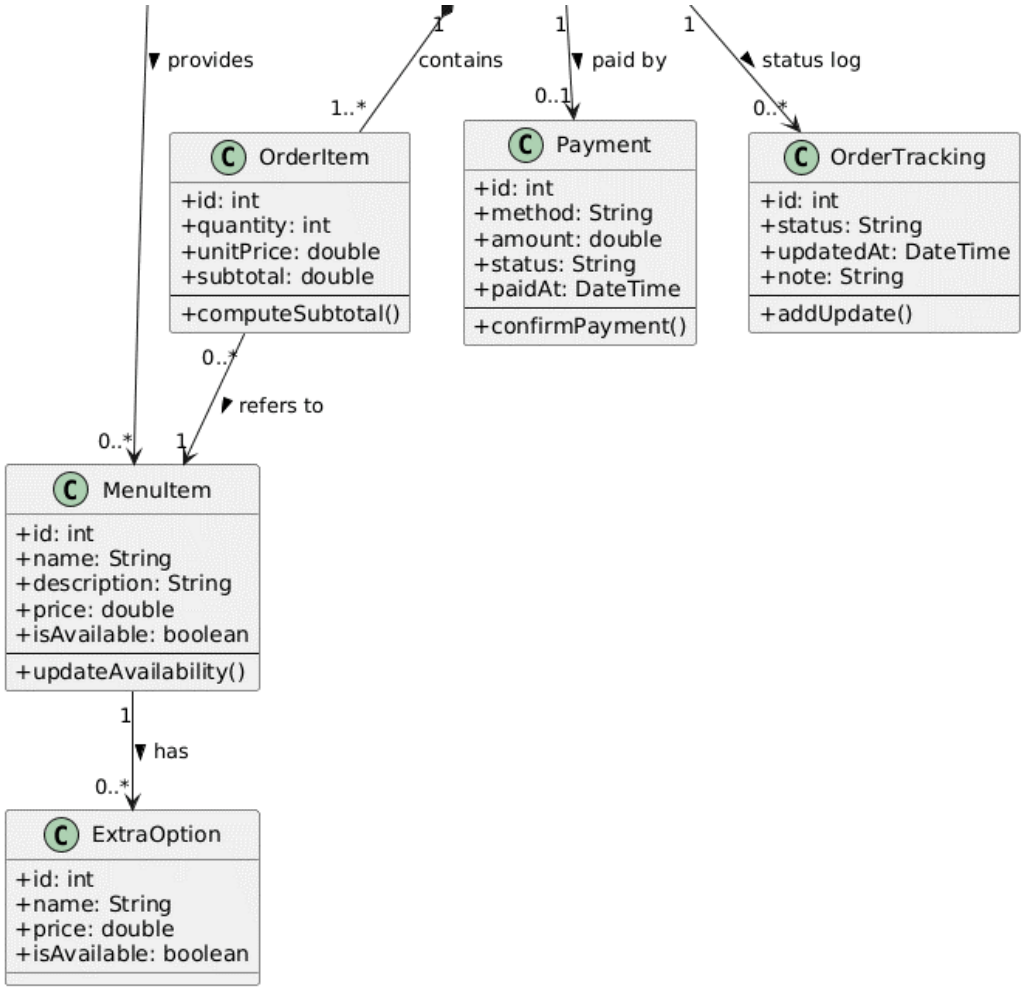


Figure 3.1: Class Diagram of the System

Class Diagram

The class diagram illustrates the main structure of the *FastBites Food Ordering and Delivery System* by showing the key system classes, their attributes, and the relationships between them. It provides an overall view of how different components interact to support the food ordering and delivery process.

The system includes core classes such as **Customer**, **Restaurant**, **Order**, **MenuItem**, **OrderItem**, **Payment**, **DeliveryDriver**, **OrderTracking**, **ExtraOption**, and **Admin**. The **Customer** places orders, while the **Restaurant** provides menu items and receives orders. Each order may include multiple order items, which are linked to specific menu items.

The **Order** class represents the central entity that connects customers, restaurants, payments, delivery drivers, and tracking information. Payment details are handled through the **Payment** class, and order status updates are recorded using the **OrderTracking** class. The **DeliveryDriver** class manages the delivery process, while **ExtraOption** allows additional customization of menu items.

The **Admin** class is responsible for managing users, restaurants, and delivery drivers. Overall, the class diagram clearly defines the system structure and ensures proper organization and interaction between system components.

1.2 Sequence Diagram

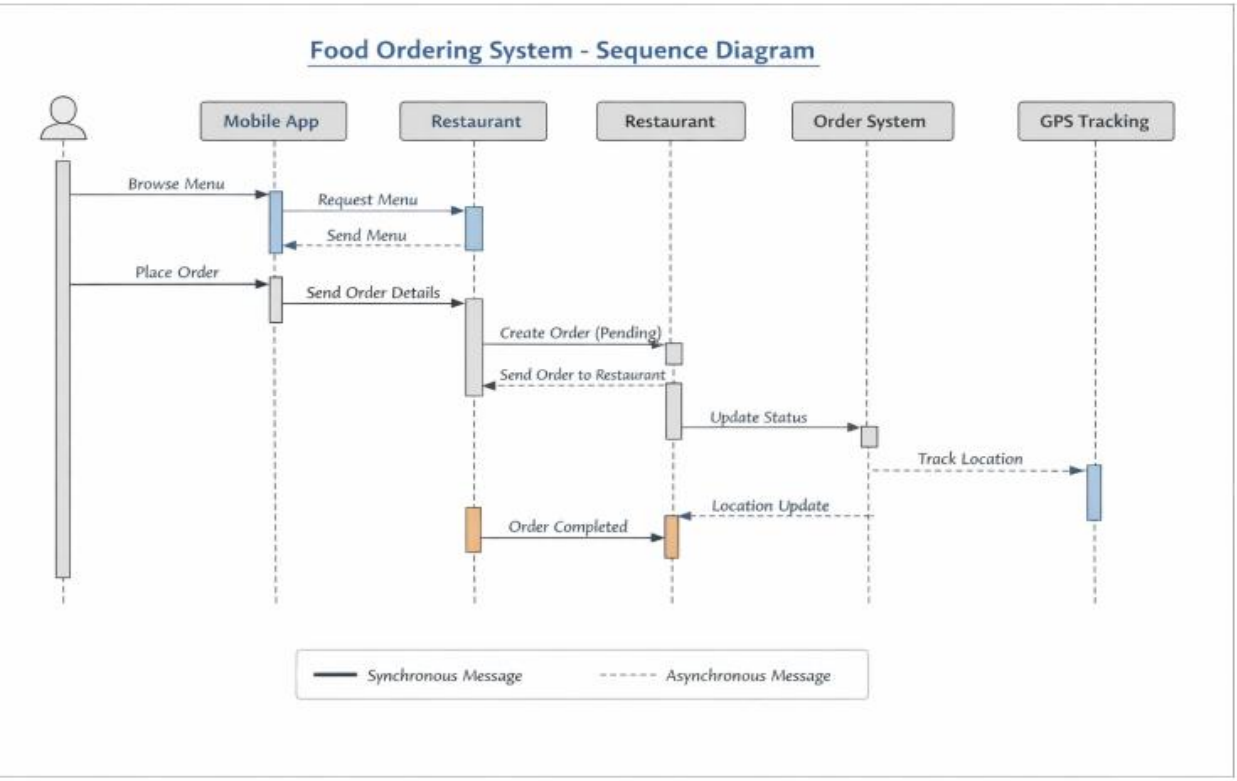


Figure 3.2: Sequence Diagram for Food Ordering Process

The sequence diagram illustrates the interaction between the main components of the food ordering system and describes how a food order is placed and processed.

The sequence begins when the user opens the mobile application and browses the available menu. The mobile application sends a request to the restaurant to retrieve the menu, and the restaurant responds by sending the available menu items.

After selecting the desired meals, the user places an order through the mobile application. The application then sends the order details to the order system, which creates a new order and assigns an initial status such as *Pending*. The order details are subsequently forwarded to the restaurant for preparation.

Once the order is accepted by the restaurant, the order status is updated accordingly. During the delivery process, the GPS tracking service continuously updates the order location.

These location updates are sent back to the order system and displayed to the user through the mobile application, allowing real-time order tracking.

Finally, when the order is completed and delivered, the restaurant updates the order status to *Completed*, and the user receives a notification through the mobile application.

This sequence diagram clearly demonstrates the flow of interactions between the user, mobile application, restaurant, order system, and GPS tracking service throughout the food ordering and delivery process.

1.3 Activity Diagram

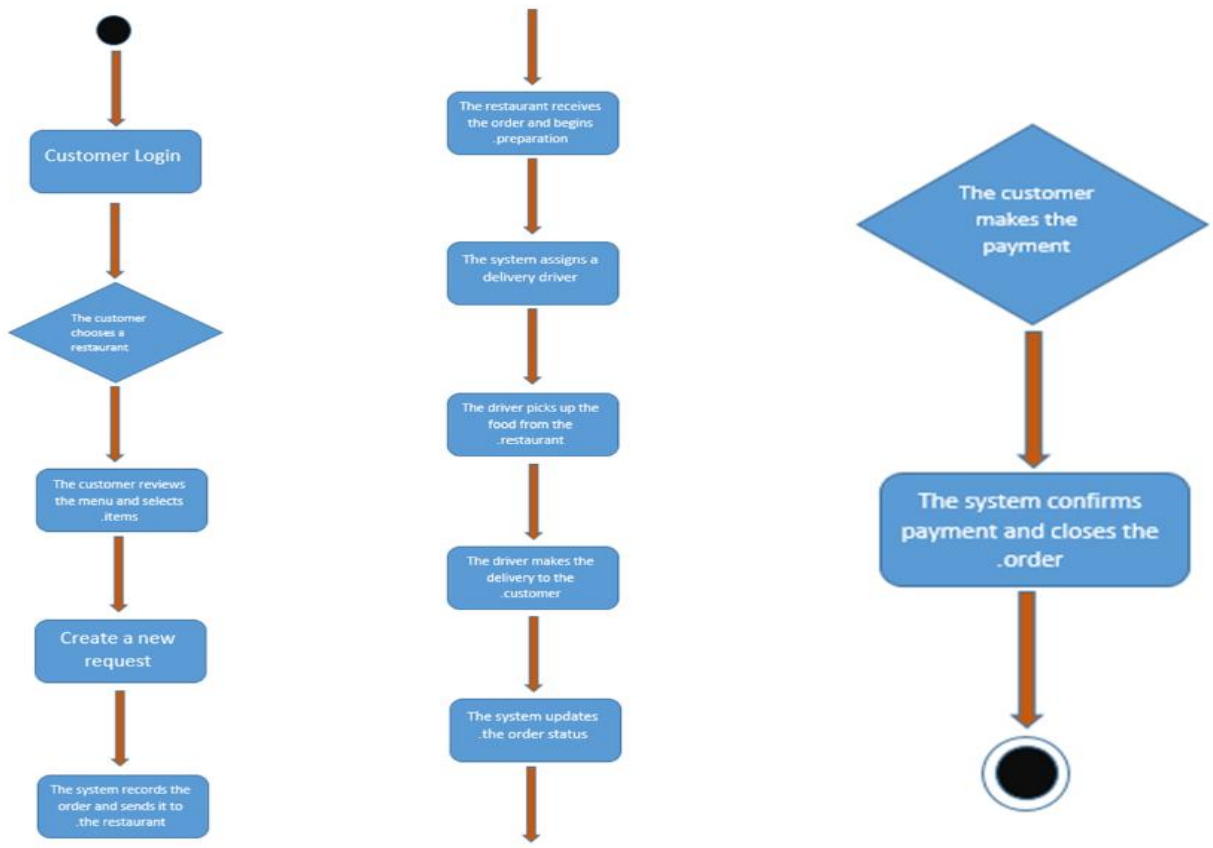


Figure 3.3: Activity Diagram

This activity diagram illustrates the complete workflow of the food delivery application, starting from customer login and ending with order closure after payment.

The process begins when the customer logs into the system. The customer then selects a preferred restaurant and reviews the available menu items. After choosing the desired food items, the system creates a new order based on the customer's selections.

Next, the system records the order and sends it to the selected restaurant. The restaurant receives the order and begins preparing the food. Once the order is ready, the system assigns a suitable delivery driver.

The driver picks up the food from the restaurant and delivers it to the customer's address. During the delivery process, the system continuously updates the order status, such as Preparing, On the Way, and Delivered.

After receiving the order, the customer completes the payment using an available payment method (cash, card, or electronic payment). Finally, the system confirms the payment and closes the order.

This activity diagram clearly demonstrates the sequential flow of actions and decisions involved in the food ordering and delivery process.

1.4 Entity Relationship Diagram (ERD)

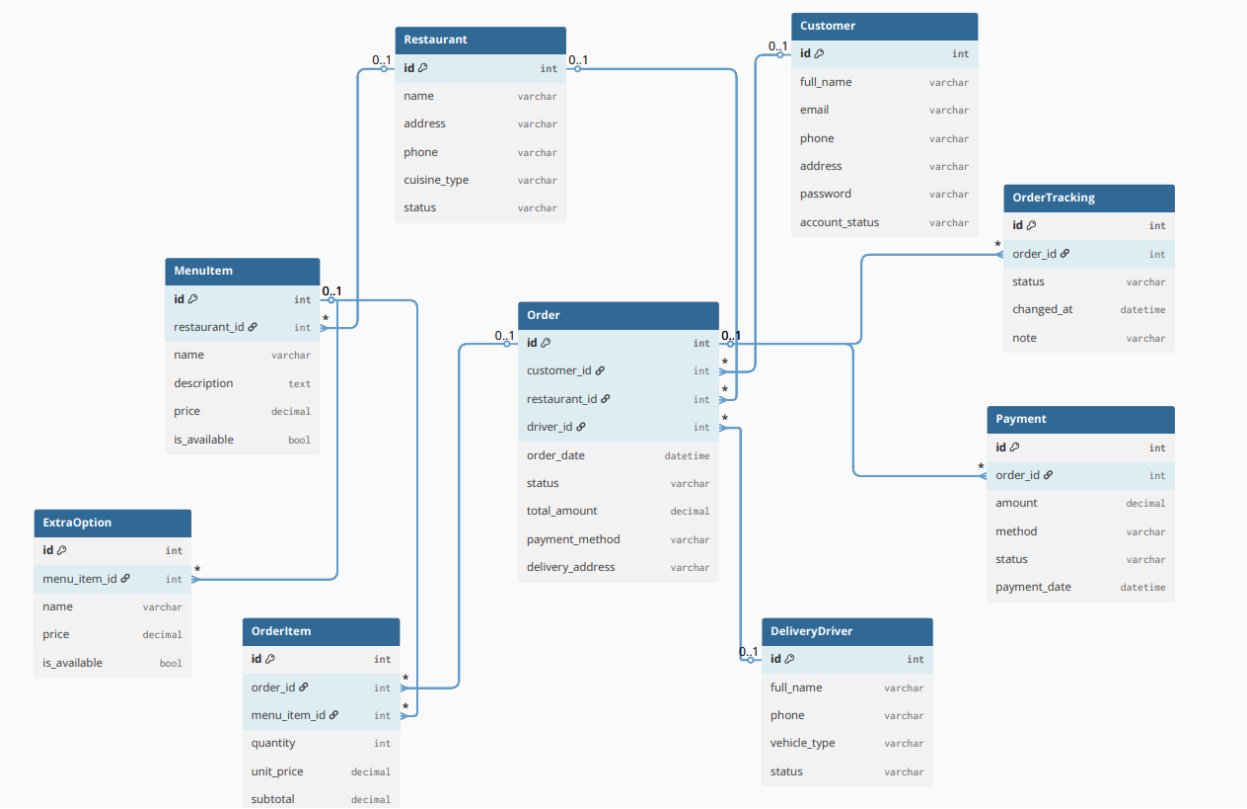


Figure 3.4: Entity Relationship Diagram (ERD)

Figure 3.4 presents the Entity Relationship Diagram (ERD) of the proposed smart food ordering and delivery system. The ERD illustrates the structure of the database, including the main entities, their attributes, primary keys, foreign keys, and the relationships between them.

The Customer entity stores customer-related information such as full name, email, phone number, address, password, and account status. Each customer can place multiple orders, which establishes a one-to-many relationship between Customer and Order.

The Restaurant entity represents restaurants registered in the system and contains details such as name, address, phone number, cuisine type, and status. A restaurant can offer multiple menu items, forming a one-to-many relationship between Restaurant and MenuItem.

The MenuItem entity stores food items offered by restaurants, including name, description, price, and availability. Each menu item belongs to a specific restaurant and can be associated with multiple order items.

The Order entity represents customer orders and includes attributes such as order date, status, total amount, payment method, and delivery address. Each order is linked to one customer, one restaurant, and optionally one delivery driver.

The OrderItem entity acts as a junction table between Order and MenuItem. It records the quantity, unit price, and subtotal for each menu item within an order, enabling the system to handle orders containing multiple items.

The ExtraOption entity represents additional options or add-ons for menu items, such as extra cheese or sauces. Each extra option is linked to a specific menu item.

The DeliveryDriver entity stores information related to delivery drivers, including full name, phone number, vehicle type, and status. A delivery driver can be assigned to multiple orders.

The Payment entity records payment details for orders, including payment amount, method, status, and payment date. Each payment is associated with a specific order.

The OrderTracking entity is used to track the status of orders over time. It stores the current status, timestamp of the update, and optional notes, allowing real-time monitoring of the delivery process.

Overall, the ERD ensures data integrity and efficient data organization by clearly defining relationships between entities and enforcing primary and foreign key constraints. This structured database design supports order management, delivery tracking, payment processing, and personalized recommendations within the system.

2. System Architecture

The proposed food ordering and delivery platform system architecture structure. This architecture outlines how system components interact with database entities, such as users, restaurants, menu items, orders, and order items. The architecture consists of four main components:

1. Presentation Layer

This layer provides all user interfaces and handles user interactions with the system. It includes:

Customer Application: Allows users to browse restaurants, view menus, place orders, receive recommendations, and track order status.

Restaurant Dashboard: Enables restaurant owners to manage their menu items, update availability, and track incoming orders.

Delivery Driver Application (optional): Used by delivery drivers to view assigned orders and update delivery status.

This layer communicates with the backend through secured REST APIs.

2. Application / Business Logic Layer

This core layer processes system operations and enforces business rules. It interacts directly with the database and manages the system's main functionalities. It includes:

Order Management Service: Handles order creation, validation, status updates, and links each order with its corresponding user, restaurant, and ordered items.

User & Restaurant Management Service: Performs authentication, registration, profile updates, and manages restaurant data.

Recommendation Engine: Uses order history and menu item information to generate personalized food and restaurant recommendations for users.

API Gateway: Acts as a unified entry point to the backend, routing front-end requests to the appropriate services.

3. Data Layer

This layer contains the relational database design. It stores all system data, including:

- Users
- Restaurants
- Menu Items
- Orders
- Order Items

Additional entities such as payments or tracking logs may also be stored in this layer (if implemented).

The data layer ensures integrity, security, and reliable access to all stored information. Both the backend logic and the recommendation engine rely on this layer for reading and writing data.

4. Recommendation Engine Component

Although technically part of the backend, this component is treated as a standalone module due to its importance. It analyzes data from Orders, Order Items, and Menu Items to learn user preferences and provide intelligent recommendations. It exposes dedicated APIs that the Customer App uses to display personalized suggestions.

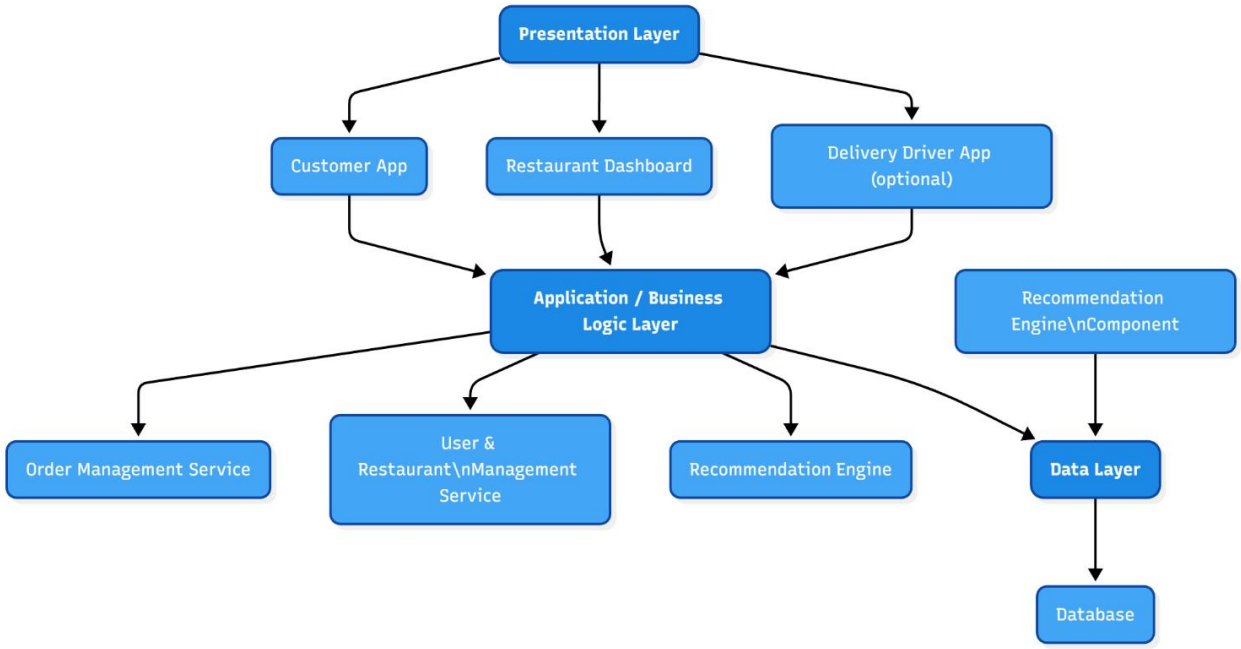


Figure 3.5: System Architecture of the Food Ordering and Delivery System

3. Database Design

The database used in this project is named *FastBites*. It is designed to support the main functionalities of the food ordering and delivery system. The database contains multiple tables that store information related to customers, drivers, restaurants, menu items, orders, payments, and additional options.

Tables and Their Purpose

- **Customer**: Stores customer information such as customer ID, name, phone number, and address.
- **Driver**: Stores delivery driver details including driver ID, name, phone number, and vehicle number.
- **Restaurant**: Contains restaurant information such as restaurant ID, name, and restaurant type (e.g., fast food, Italian).

- **MenuItem**: Stores menu items offered by restaurants, including item name, price, and restaurant ID. Each menu item is linked to a specific restaurant.
- **Order**: Represents customer orders and includes order date, total amount, and references to the customer, driver, and restaurant.
- **OrderTracking**: Tracks the status of each order (e.g., pending, on the way, delivered) along with the last update time.
- **Payment**: Stores payment details for orders, including payment method, amount, and payment date.
- **OrderDetail**: Stores detailed information about items included in each order, such as quantity, unit price, and subtotal.
- **Extra**: Represents additional options or add-ons for menu items, such as extra cheese or sauces.

Database Relationships

- A customer can place multiple orders.
- A driver can deliver multiple orders.
- A restaurant can provide multiple menu items.
- An order can contain multiple menu items through the OrderDetail table.
- Each order has associated payment and tracking records.
- Menu items can have multiple extra options.

This database design ensures data integrity and supports efficient order management, delivery tracking, and payment processing within the system.

4. User Interface Design

This section describes the user interface design of the proposed food ordering and delivery system. The system provides different interfaces tailored to the needs of various user roles, including customers, administrators, managers, and restaurant staff. Each interface is designed to be user-friendly, intuitive, and efficient in supporting system operations.

4.1 Customer Application Interfaces

The customer application provides a set of interactive screens that allow users to browse restaurants, place orders, and track deliveries easily.

- **Home Screen**
  The home screen is the first interface displayed when the application is opened. It presents promotional banners, special offers, and quick navigation options to access menus, reservations, and contact information.
- **Menu Interface**
  This interface displays all available food and drink items categorized into sections such as appetizers, main dishes, desserts, and beverages. Each item includes a name, image, price, and brief description.
- **Item Details Screen**
  The item details screen provides detailed information about selected menu items, including ingredients, customization options, and additional notes to help customers make informed decisions.
- **Cart / Order Summary**
  The cart interface allows customers to review selected items, update quantities, remove items, and view the total order cost before checkout.
- **Checkout Interface**
  This screen enables customers to enter delivery or pickup details, select a payment method, and confirm the order.

- **Profile / Account Screen**

  The profile screen allows users to manage personal information, view order history, save favorite items, and update delivery or payment details.

- **Order Tracking Screen**

  This interface displays the current order status (e.g., preparing, on the way, delivered) and provides real-time updates to enhance transparency.

- **Contact Us Interface**

  The contact screen provides restaurant contact details, including phone number, email, location map, and social media links.

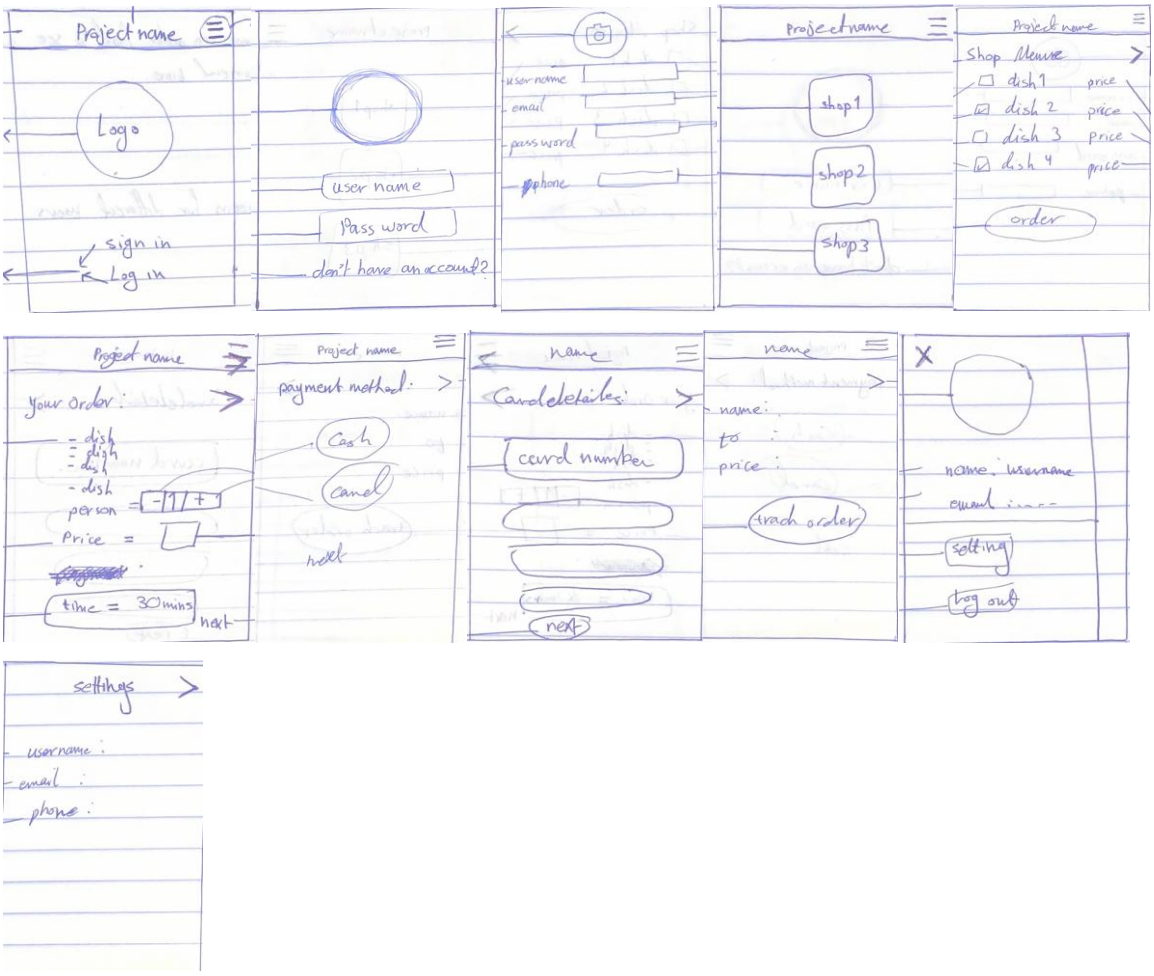


Figure 3.6: Initial User Interface Wireframes for the Customer Application

4.2 Admin and Super Admin Interfaces

Administrative interfaces are designed to support monitoring, reporting, and system management.

- **Admin Dashboard**
  The admin dashboard displays key performance indicators (KPIs) such as total revenue, total orders, active customers, and average order value. It also includes visual charts such as weekly revenue reports and daily order statistics.
- **Super Admin Dashboard**
  The super admin dashboard provides system-wide insights through advanced analytics, including sales distribution charts, weekly revenue comparisons, and payment method analysis. A table of recently placed orders enables real-time operational monitoring.

Figure 3.7: Initial Wireframes for Admin and Super Admin Interfaces

4.3 Manager Interface

The manager interface supports operational supervision and user management.

- **Login Screen**
  Provides secure access for managers using email and password authentication.
- **Dashboard Screen**
  Displays a summary of system statistics such as total orders, total earnings, active shops, and recent order activity.
- **Manage Orders**
  Allows managers to view, track, and manage all orders in the system.
- **Manage Users**
  Enables control over user accounts and shops, including adding, editing, or deleting records.
- **Reports and Settings**
  Provides access to sales reports and system configuration options.

4.4 Restaurant and Kitchen Staff Interface

This interface is designed for restaurant owners and kitchen staff to manage food preparation and order processing.

- **Login Screen**
  Ensures secure access for restaurant staff.
- **Kitchen Dashboard**
  Displays orders categorized by status, such as new orders, orders in preparation, ready for delivery, and completed orders.
- **Order Management Sections**
  Allows staff to view order details, track preparation time, and update order status.
- **Settings**
  Enables restaurant administrators to manage restaurant details, categories, working hours, and account settings.

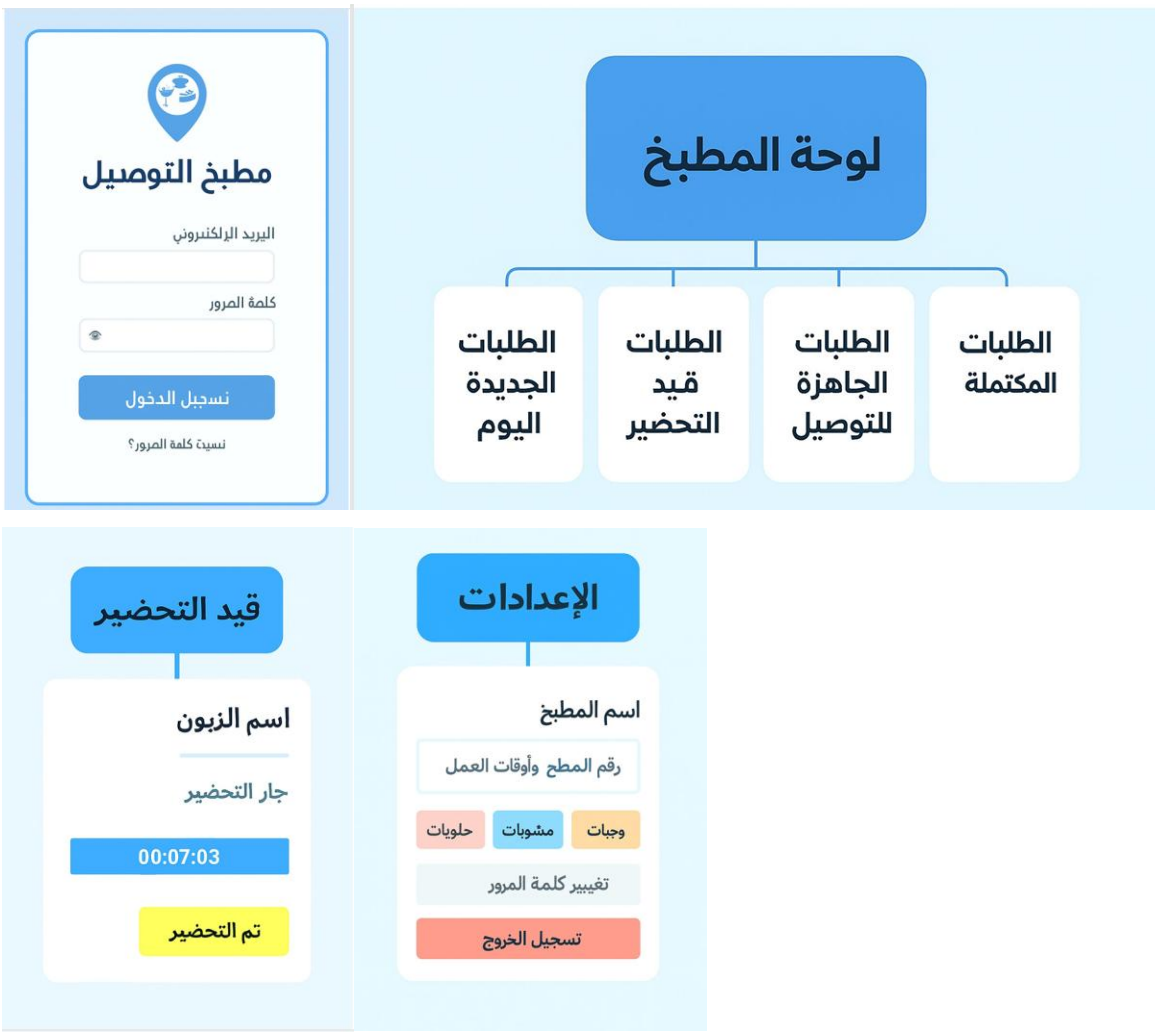


Figure 3.8: User Interface Wireframes for Restaurant and Kitchen Staff Operations

5. Development Environment

🖥️ Development Environment Table

Component	Technology / Tools	Purpose
Frontend	React.js / Angular (Web), Flutter / React Native (Mobile)	Build customer interface for browsing restaurants, menus, placing orders, and tracking deliveries
APIs	RESTful APIs / GraphQL	Communication between frontend and backend (e.g., `/restaurants`, `/orders`, `/tracking`)

Backend	Node.js + Express / Django / Flask	Handle business logic, order management, restaurant coordination, driver assignment, and payments
Database	MySQL / PostgreSQL	Store structured data: Customers, Drivers, Restaurants, Menu Items, Orders, Payments, Tracking

Order Management	Backend Logic (Node.js/ Django) + Database	Create orders, assign drivers, update status, send notifications
Payment Management	Payment Gateway (Stripe, PayPal, Local APIs)	Handle multiple payment methods, confirm transactions, generate
Monitoring	Prometheus & Grafana / Firebase Analytics	Monitor performance, track errors, and analyze user behavior
Application Interfaces	Flutter UI / React Native Components	Build customer, driver, and restaurant interfaces (login, menus, cart, tracking, payments)

Deployment	Docker + Kubernetes / AWS, Azure, GCP	Host and scale the application in the cloud
CI/CD	GitHub Actions / Jenkins / GitLab CI	Automate testing and deployment pipelines
Version Control	Git + GitHub/ GitLab	Manage source code and team collaboration
Testing	Jest, Mocha (Backend), Cypress (Frontend)	Ensure reliability and quality of the application

Figure 3.9: Development Environment Tools and Technologies

6. Tools, Languages, and Frameworks

This project utilizes a set of modern tools, programming languages, and frameworks to develop a scalable and efficient food ordering and recommendation platform.

1. Programming Languages

- **JavaScript**

Used primarily for building the frontend interface and backend services through Node.js.

- **Python**

Used for implementing the recommendation engine due to its strong ecosystem for machine learning and data analysis.

2. Frameworks and Libraries

- **Backend Frameworks**

Node.js (Express.js): Used to build RESTful APIs, handle user authentication, process orders, and connect to the database.

- **Frontend Frameworks**

React.js (or Flutter Web/App): Used to build responsive and interactive interfaces for the customer application and restaurant dashboard.

- **Machine Learning Libraries (Python)**

Scikit-learn: For building recommendation models such as collaborative filtering and content-based recommendation.

NumPy & Pandas: For data cleaning, processing, and feature extraction.

3. Database Technologies

MySQL: A relational database management system used to store structured data such as users, restaurants, menu items, and orders.

4. Development Tools

Visual Studio Code: Main development environment for frontend, backend, and machine learning scripts.

Postman: Used to test and validate API endpoints during backend development.

Git & GitHub: Version control and collaboration tools used for managing the codebase.

Figma: Used for designing the prototype (UI/UX) of the system's applications.

CHAPTER FOUR

CONCLUSION

1. Conclusion

This project presented the design and implementation of a smart food ordering and delivery platform that allows users to browse restaurants, select meals, place orders, and track their order status in real time through an easy-to-use and interactive system. The proposed platform aims to simplify the food ordering process while improving coordination and communication between customers, restaurants, and delivery drivers.

The system integrates an intelligent recommendation engine that suggests meals and restaurants based on user preferences and previous ordering behavior, which enhances the overall user experience. In addition, a GPS-based tracking mechanism was implemented to allow customers to monitor delivery progress and view order statuses such as *Pending*, *In Progress*, and *Completed*, increasing transparency and user trust.

During the development process, UML diagrams including Class Diagrams, Sequence Diagrams, and Activity Diagrams were used to model the system structure and workflows clearly. The database was also carefully designed to efficiently store and manage system data, supporting reliable order processing, secure payment handling, and accurate delivery tracking.

In conclusion, the developed platform demonstrates an efficient and user-centered approach to modern food delivery applications. The project successfully meets its objectives by providing a reliable and practical solution for food ordering and delivery.

References

1. Sommerville, I., Software Engineering, 10th Edition, Pearson Education, 2016.

2. Object Management Group (OMG), Unified Modeling Language (UML) Documentation.

3. Android Developers, Android Application Development Documentation.

4. Firebase Documentation, Authentication and Realtime Database.

5. Google Developers, Location and GPS Services Documentation.

Appendices

Appendix A: UML Diagrams

This appendix contains the UML diagrams used in the project, including the Class Diagram, Sequence Diagram, and Activity Diagram.

Appendix B: User Interface Snapshots

This appendix includes screenshots of the main user interfaces of the application, such as the login screen, restaurant list, meal list, order summary, and order tracking screen.

Appendix C: Database Design

This appendix presents the database structure and tables used in the system, including Customers, Restaurants, Orders, Menu Items, Payments, and Tracking tables.

Appendix D: Code Samples

This appendix includes selected code samples that demonstrate the core functionalities of the system.

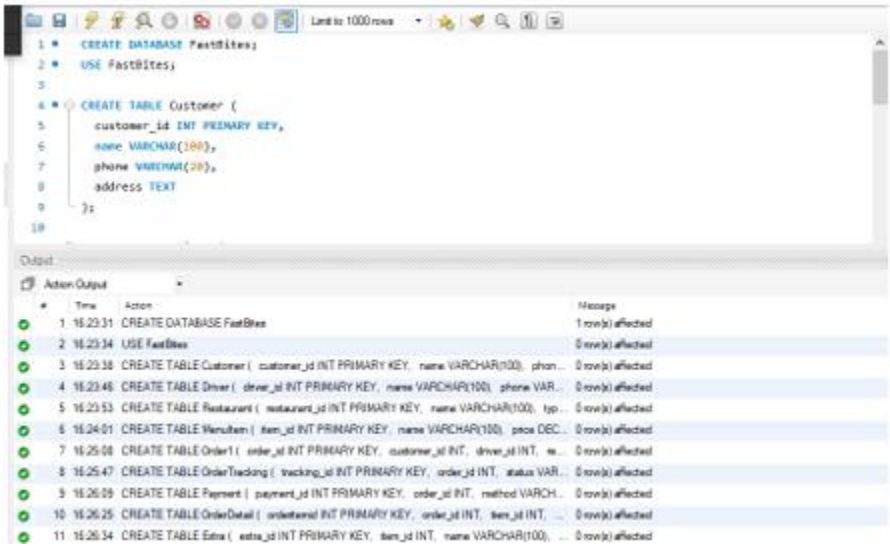


Figure D.1: SQL Database Creation and Execution Log

```
CREATE DATABASE FastBites;

USE FastBites;

CREATE TABLE Customer (

    customer_id INT PRIMARY KEY,

    name VARCHAR(100),

    phone VARCHAR(20),

    address TEXT

);


CREATE TABLE Driver (

    driver_id INT PRIMARY KEY,
```

```
    name VARCHAR(100),
    phone VARCHAR(20),
    vehicle_number VARCHAR(20)
);

CREATE TABLE Restaurant (
    restaurant_id INT PRIMARY KEY,
    name VARCHAR(100),
    type VARCHAR(50)
);

CREATE TABLE MenuItem (
    item_id INT PRIMARY KEY,
    name VARCHAR(100),
    price DECIMAL(6,2),
    restaurant_id INT,
    FOREIGN KEY (restaurant_id) REFERENCES Restaurant(restaurant_id)
);

CREATE TABLE Order1 (
    order_id INT PRIMARY KEY,
    customer_id INT,
    driver_id INT,
    restaurant_id INT,
```

```
    order_date DATETIME,
    total_amount DECIMAL(8,2),
    FOREIGN KEY (customer_id) REFERENCES Customer(customer_id),
    FOREIGN KEY (driver_id) REFERENCES Driver(driver_id),
    FOREIGN KEY (restaurant_id) REFERENCES Restaurant(restaurant_id)
);

CREATE TABLE OrderTracking (
    tracking_id INT PRIMARY KEY,
    order_id INT,
    status VARCHAR(50),
    updated_at DATETIME,
    FOREIGN KEY (order_id) REFERENCES Order1(order_id)
);

CREATE TABLE Payment (
    payment_id INT PRIMARY KEY,
    order_id INT,
    method VARCHAR(50),
    amount DECIMAL(8,2),
    paid_at DATETIME,
    FOREIGN KEY (order_id) REFERENCES Order1(order_id)
);
```

```
CREATE TABLE OrderDetail (
    orderitemid INT PRIMARY KEY,
    order_id INT,
    item_id INT,
    quantity INT,
    unit_price DECIMAL(6,2),
    subtotal DECIMAL(8,2),
    FOREIGN KEY (order_id) REFERENCES Order1(order_id),
    FOREIGN KEY (item_id) REFERENCES MenuItem(item_id)
);


CREATE TABLE Extra (
    extra_id INT PRIMARY KEY,
    item_id INT,
    name VARCHAR(100),
    price DECIMAL(6,2),
    FOREIGN KEY (item_id) REFERENCES MenuItem(item_id)
);
```